

✓ Lab 3: Multi-Source Retail Sales Data Integration and Analysis

Duration: 2 Hours

Platform: R / Google Colab

Domain: Retail Analytics

Dataset: UCI Online Retail Dataset

Objective: Import, clean, integrate, and analyze heterogeneous retail datasets (CSV, JSON, Excel) and store the final processed data in an SQLite database.

✓ Step 1: Install and Load Required Packages

```
install.packages(c(
  "tidyverse",
  "jsonlite",
  "readxl",
  "DBI",
  "RSQLite",
  "writexl"
))
```

Installing packages into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

```
# Load required libraries
library(tidyverse)
library(jsonlite)
library(readxl)
library(DBI)
library(RSQLite)
library(writexl)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1    ✓ readr      2.2.0
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2     4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr       1.2.2

— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
! Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

Attaching package: 'jsonlite'

The following object is masked from 'package:purrr':

  flatten
```

✓ Task 1: Import and Clean the Data

Step 2: Read the Original Dataset

```
getwd()

retail <- read_excel("Online Retail.xlsx")

head(retail)
dim(retail)
str(retail)
colnames(retail)
```

'/content'

A tibble: 6 × 8

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
<chr>	<chr>	<chr>	<dbl>	<dtm>	<dbl>	<dbl>	<chr>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00	7.65	17850	United Kingdom

541909 × 8

tibble [541,909 × 8] (S3: tbl_df/tbl/data.frame)

\$ InvoiceNo : chr [1:541909] "536365" "536365" "536365" "536365" ...

\$ StockCode : chr [1:541909] "85123A" "71053" "84406B" "84029G" ...

\$ Description: chr [1:541909] "WHITE HANGING HEART T-LIGHT HOLDER" "WHITE METAL LANTERN" "CREAM CUPID HEARTS COAT HANGER" "

Step 3: Inspect Data Quality Issues

```
# Check missing values
colSums(is.na(retail))

# Check duplicate rows
sum(duplicated(retail))

# Check invalid quantities
sum(retail$Quantity <= 0, na.rm = TRUE)

# Check invalid unit prices
sum(retail$UnitPrice <= 0, na.rm = TRUE)
```

```
InvoiceNo:    0 StockCode:    0 Description:    1454 Quantity:    0 InvoiceDate:    0 UnitPrice:    0 CustomerID:    135080
Country:      0
5268
10624
```

Step 4: Clean the Dataset

Cleaning Decisions:

1. **Rows with missing CustomerID** are removed because customer identification is essential for customer-level analysis and segmentation.
2. **Rows with missing Description** are removed as they indicate incomplete product records that cannot be meaningfully analyzed.
3. **Rows with Quantity ≤ 0** are removed because zero or negative quantities represent cancellations or returns, not valid sales transactions.
4. **Rows with UnitPrice ≤ 0** are removed because zero or negative prices indicate free items or data errors, which would distort revenue calculations.
5. **Duplicate rows** are removed using `distinct()` to avoid inflating metrics.

```
retail_clean <- retail %>%
  filter(
    !is.na(CustomerID),
    !is.na(Description),
    Quantity > 0,
    UnitPrice > 0
  ) %>%
  distinct()

# Check dimensions after cleaning
dim(retail_clean)

# View cleaned data
head(retail_clean)
```

392692 · 8

A tibble: 6 × 8

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
<chr>	<chr>	<chr>	<dbl>	<dtm>	<dbl>	<dbl>	<chr>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom

Verify cleaning was successful

Verify missing values
colSums(is.na(etail_clean))# Verify invalid quantities
sum(etail_clean\$Quantity <= 0)# Verify invalid prices
sum(etail_clean\$UnitPrice <= 0)# Verify duplicates
sum(duplicated(etail_clean))# Final dimensions
dim(etail_clean)

InvoiceNo: 0 StockCode: 0 Description: 0 Quantity: 0 InvoiceDate: 0 UnitPrice: 0 CustomerID: 0 Country:

0

0

0

Step 5: Create Revenue Attribute

Revenue = Quantity × UnitPrice

```
# Calculate Revenue
etail_clean <- etail_clean %>%
  mutate(
    Revenue = Quantity * UnitPrice
  )
```

```
# View cleaned data with Revenue
head(etail_clean)
```

A tibble: 6 × 9

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Revenue
<chr>	<chr>	<chr>	<dbl>	<dtm>	<dbl>	<dbl>	<chr>	<dbl>
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom	15.30
536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	20.34
536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850	United Kingdom	22.00
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	20.34

Step 6: Split Data into Three Separate Files

Organizing the data into:

- **transactions.csv** – InvoiceNo, StockCode, CustomerID, Quantity, InvoiceDate
- **products.json** – StockCode, Description, UnitPrice
- **customers.xlsx** – CustomerID, Country

```
# Create transactions dataset
transactions <- retail_clean %>%
  select(
    InvoiceNo,
    StockCode,
    CustomerID,
    Quantity,
    InvoiceDate
  )

# Export to CSV
write_csv(transactions, "transactions.csv")

# Check
head(transactions)
```

A tibble: 6 × 5

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate
<chr>	<chr>	<dbl>	<dbl>	<dtm>
536365	85123A	17850	6	2010-12-01 08:26:00
536365	71053	17850	6	2010-12-01 08:26:00
536365	84406B	17850	8	2010-12-01 08:26:00
536365	84029G	17850	6	2010-12-01 08:26:00
536365	84029E	17850	6	2010-12-01 08:26:00
536365	22752	17850	2	2010-12-01 08:26:00

```
# Create products dataset
products <- retail_clean %>%
  select(
    StockCode,
    Description,
    UnitPrice
  ) %>%
  distinct(StockCode, .keep_all = TRUE)

# Export to JSON
write_json(
  products,
  "products.json",
  pretty = TRUE,
  auto_unbox = TRUE
)

# Check
head(products)
```

A tibble: 6 × 3

StockCode	Description	UnitPrice
<chr>	<chr>	<dbl>
85123A	WHITE HANGING HEART T-LIGHT HOLDER	2.55
71053	WHITE METAL LANTERN	3.39
84406B	CREAM CUPID HEARTS COAT HANGER	2.75
84029G	KNITTED UNION FLAG HOT WATER BOTTLE	3.39
84029E	RED WOOLLY HOTTIE WHITE HEART.	3.39
22752	SET 7 BABUSHKA NESTING BOXES	7.65

```
# Create customers dataset
customers <- retail_clean %>%
  select(
    CustomerID,
    Country
  ) %>%
  distinct(CustomerID, .keep_all = TRUE)

# Export to Excel
write_xlsx(customers, "customers.xlsx")

# Check
head(customers)
```

A tibble: 6 × 2

CustomerID	Country
<dbl>	<chr>
17850	United Kingdom
13047	United Kingdom
12583	France
13748	United Kingdom
15100	United Kingdom
15291	United Kingdom

Task 2: Integrate the Multiple Data Sources

Step 7: Import the Three Data Sources

```
# Import CSV
transactions <- read_csv("transactions.csv")

# Import JSON
products <- fromJSON("products.json")

# Import Excel
customers <- read_excel("customers.xlsx")

# Inspect the datasets
head(transactions)
head(products)
head(customers)
```

Rows: 392692 Columns: 5

— Column specification —

Delimiter: ","

chr (1): StockCode

dbl (3): InvoiceNo, CustomerID, Quantity

dtm (1): InvoiceDate

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

A tibble: 6 × 5

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate
<dbl>	<chr>	<dbl>	<dbl>	<dtm>
536365	85123A	17850	6	2010-12-01 08:26:00
536365	71053	17850	6	2010-12-01 08:26:00
536365	84406B	17850	8	2010-12-01 08:26:00
536365	84029G	17850	6	2010-12-01 08:26:00
536365	84029E	17850	6	2010-12-01 08:26:00
536365	22752	17850	2	2010-12-01 08:26:00

A data.frame: 6 × 3

StockCode	Description	UnitPrice
<chr>	<chr>	<dbl>
1 85123A	WHITE HANGING HEART T-LIGHT HOLDER	2.55
2 71053	WHITE METAL LANTERN	3.39
3 84406B	CREAM CUPID HEARTS COAT HANGER	2.75
4 84029G	KNITTED UNION FLAG HOT WATER BOTTLE	3.39
5 84029E	RED WOOLLY HOTTIE WHITE HEART.	3.39
6 22752	SET 7 BABUSHKA NESTING BOXES	7.65

A tibble: 6 × 2

CustomerID	Country
<dbl>	<chr>
17850	United Kingdom
13047	United Kingdom
12583	France
13748	United Kingdom
15100	United Kingdom
15291	United Kingdom

Check dimensions

dim(transactions)

dim(products)

dim(customers)

Check key columns

colnames(transactions)

colnames(products)

colnames(customers)

392692 · 5

3665 · 3

4338 · 2

'InvoiceNo' · 'StockCode' · 'CustomerID' · 'Quantity' · 'InvoiceDate'

'StockCode' · 'Description' · 'UnitPrice'

'CustomerID' · 'Country'

Step 8: Integrate the Three Datasets

Justification for using `left_join()`:

`left_join` is chosen over `inner_join` to retain **ALL** transaction records from the primary transactions dataset, even if a matching product or customer record is not found. This allows us to:

1. **Preserve the complete transaction history** without data loss.
2. **Identify and investigate unmatched records** (NAs) after the join.
3. **Understand data quality issues** across the different source systems.

Using `inner_join` would silently drop unmatched transactions, potentially losing valuable sales data and masking data integration problems.

```
# Join transactions with products
sales_with_products <- transactions %>%
  left_join(products, by = "StockCode")

# Join customer information
retail_final <- sales_with_products %>%
  left_join(customers, by = "CustomerID")

# Check final dimensions
dim(retail_final)

# View the integrated data
head(retail_final)
```

392692 · 8

A tibble: 6 × 8

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate	Description	UnitPrice	Country
<dbl>	<chr>	<dbl>	<dbl>	<dtm>	<chr>	<dbl>	<chr>
536365	85123A	17850	6	2010-12-01 08:26:00	WHITE HANGING HEART T-LIGHT HOLDER	2.55	United Kingdom
536365	71053	17850	6	2010-12-01 08:26:00	WHITE METAL LANTERN	3.39	United Kingdom
536365	84406B	17850	8	2010-12-01 08:26:00	CREAM CUPID HEARTS COAT HANGER	2.75	United Kingdom
536365	84029G	17850	6	2010-12-01 08:26:00	KNITTED UNION FLAG HOT WATER BOTTLE	3.39	United Kingdom

```
# Check unmatched product records
sum(is.na(sales_with_products$Description))

# Check unmatched customer records
sum(is.na(retail_final$Country))

# Check missing values in final dataset
colSums(is.na(retail_final))
```

0

0

InvoiceNo: 0 StockCode: 0 CustomerID: 0 Quantity: 0 InvoiceDate: 0 Description: 0 UnitPrice: 0 Country:

Step 9: Calculate Revenue in the Integrated Dataset

```
retail_final <- retail_final %>%
  mutate(
    Revenue = Quantity * UnitPrice
  )

# Check
head(retail_final)

# Check dimensions
dim(retail_final)
```

A tibble: 6 × 9

InvoiceNo	StockCode	CustomerID	Quantity	InvoiceDate	Description	UnitPrice	Country	Revenue
<dbl>	<chr>	<dbl>	<dbl>	<dtm>	<chr>	<dbl>	<chr>	<dbl>
536365	85123A	17850	6	2010-12-01 08:26:00	WHITE HANGING HEART T-LIGHT HOLDER	2.55	United Kingdom	15.30
536365	71053	17850	6	2010-12-01 08:26:00	WHITE METAL LANTERN	3.39	United Kingdom	20.34
536365	84406B	17850	8	2010-12-01 08:26:00	CREAM CUPID HEARTS COAT HANGER	2.75	United Kingdom	22.00
536365	84029G	17850	6	2010-12-01 08:26:00	KNITTED UNION FLAG HOT WATER BOTTLE	3.39	United Kingdom	20.34

Task 3: Perform Sales and Customer Analysis

Step 10: Total Sales Revenue

```
total_revenue <- retail_final %>%
  summarise(
    Total_Revenue = sum(Revenue)
  )

# Display total revenue in a readable format
cat(
  "Total Sales Revenue: \u00a3",
  format(round(total_revenue$Total_Revenue, 2),
    big.mark = ",",
    nsmall = 2
  ),
  "\n"
)
```

total_revenue

Total Sales Revenue: £ 9,546,218.70
A tibble: 1 × 1

Total_Revenue
9546219

Step 11: Top 5 Products by Revenue

```
top_products <- retail_final %>%
  group_by(StockCode, Description) %>%
  summarise(
    Total_Revenue = sum(Revenue),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue)) %>%
  slice_head(n = 5)
```

top_products

A tibble: 5 × 3

StockCode	Description	Total_Revenue
<chr>	<chr>	<dbl>
23843	PAPER CRAFT , LITTLE BIRDIE	168469.60
22423	REGENCY CAKESTAND 3 TIER	135495.30
85123A	WHITE HANGING HEART T-LIGHT HOLDER	93745.65
23166	MEDIUM CERAMIC TOP STORAGE JAR	81032.64
85099B	JUMBO BAG RED RETROSPOT	76028.70

Step 12: Top 5 Countries by Revenue

```
top_countries <- retail_final %>%
  group_by(Country) %>%
  summarise(
    Total_Revenue = sum(Revenue),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue)) %>%
  slice_head(n = 5)

top_countries

# Display revenue with commas
top_countries %>%
  mutate(
    Total_Revenue = round(Total_Revenue, 2)
  )
```

```
A tibble: 5 × 2
  Country Total_Revenue
  <chr>      <dbl>
United Kingdom 7868408.7
Netherlands    329130.8
EIRE           287260.9
Germany        233804.1
France         203628.3
```

```
A tibble: 5 × 2
  Country Total_Revenue
  <chr>      <dbl>
United Kingdom 7868408.7
Netherlands    329130.8
EIRE           287260.9
Germany        233804.1
France         203628.3
```

Step 13: Top 5 Customers by Total Purchase Value

```
top_customers <- retail_final %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Purchase_Value = sum(Revenue),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Purchase_Value)) %>%
  slice_head(n = 5)
```

```
top_customers
```

```
A tibble: 5 × 2
  CustomerID Total_Purchase_Value
  <dbl>      <dbl>
18102      383153.0
14646      323767.5
17450      171051.9
16446      168472.5
14911      155092.0
```

Step 14: Calculate Customer Purchase Value Distribution

```
# Calculate total purchase value for every customer
customer_value <- retail_final %>%
  group_by(CustomerID) %>%
  summarise(
    Total_Purchase_Value = sum(Revenue),
    .groups = "drop"
  )

# Examine the distribution
summary(customer_value$Total_Purchase_Value)

# Calculate useful percentiles
quantile(
  customer_value$Total_Purchase_Value,
  probs = c(0.25, 0.50, 0.75),
  na.rm = TRUE
)
```

```
      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
1.25    317.84    703.57   2200.60   1744.91 383152.99
25%:    317.835 50%:    703.57 75%:    1744.9075
```

Step 15: Classify Customers Using `case_when()`

Customers are classified into four categories based on their total purchase value using the **25th, 50th, and 75th percentile** thresholds:

Category	Threshold
Low Value	Total Purchase Value $\leq$ Q1 (25th percentile)
Medium Value	Q1 < Total Purchase Value $\leq$ Q2 (50th percentile)
High Value	Q2 < Total Purchase Value $\leq$ Q3 (75th percentile)
Premium	Total Purchase Value > Q3 (75th percentile)

```
customer_value <- customer_value %>%
  mutate(
    Customer_Category = case_when(
      Total_Purchase_Value <= 317.835 ~ "Low Value",
      Total_Purchase_Value <= 703.570 ~ "Medium Value",
      Total_Purchase_Value <= 1744.907 ~ "High Value",
      Total_Purchase_Value > 1744.907 ~ "Premium"
    )
  )

# View classified customers
head(customer_value)

# Count customers in each category
customer_value %>%
  count(Customer_Category)
```

A tibble: 6 × 3

CustomerID	Total_Purchase_Value	Customer_Category
<dbl>	<dbl>	<chr>
12346	77183.60	Premium
12347	4737.58	Premium
12348	1661.64	High Value
12349	1523.47	High Value
12350	307.48	Low Value
12352	1444.07	High Value

A tibble: 4 × 2

Customer_Category	n
<chr>	<int>
High Value	1084
Low Value	1085
Medium Value	1084
Premium	1085

```
# Revenue contribution by customer category
customer_category_summary <- customer_value %>%
  group_by(Customer_Category) %>%
  summarise(
    Number_of_Customers = n(),
    Total_Revenue = sum(Total_Purchase_Value),
    Average_Purchase_Value = mean(Total_Purchase_Value),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue))
```

customer_category_summary

A tibble: 4 × 4

Customer_Category	Number_of_Customers	Total_Revenue	Average_Purchase_Value
<chr>	<int>	<dbl>	<dbl>
Premium	1085	7599463.5	7004.1138
High Value	1084	1218830.9	1124.3828
Medium Value	1084	526193.9	485.4187
Low Value	1085	201730.5	185.9267

▼ Step 16: Country Performance Analysis — High and Underperforming Markets

```
# Complete country performance analysis
country_analysis <- retail_final %>%
  group_by(Country) %>%
  summarise(
    Total_Revenue = sum(Revenue),
    Number_of_Transactions = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(Total_Revenue))

country_analysis

# Highest revenue markets
head(country_analysis, 5)

# Lowest revenue markets
tail(country_analysis, 5)
```


A tibble: 31 × 3

Country	Total_Revenue	Number_of_Transactions
<chr>	<dbl>	<int>
United Kingdom	7868408.65	349203
Netherlands	329130.77	2359
EIRE	287260.87	7226
Germany	233804.06	9025
France	203628.28	8326
Australia	157316.33	1253
Spain	62080.85	2408
Switzerland	57268.14	1825
United Arab Emirates	2092.89	92
Saudi Arabia	145.92	9
Dominican Republic	544.00	11

Market Performance Observations

High-Performing Market: United Kingdom

The United Kingdom is the highest-performing market, contributing the vast majority of total revenue and transaction volume. This is expected as the company is UK-based, with a well-established customer base, strong brand recognition, and efficient logistics infrastructure in its home market.

Underperforming Market: Saudi Arabia (or similar low-revenue country)

Markets such as Saudi Arabia appear at the bottom of the revenue ranking with minimal transactions and negligible revenue. This indicates very limited market penetration, possibly due to lack of marketing presence, logistical challenges in shipping, or cultural/regional product-market fit issues. These markets represent potential growth opportunities if targeted strategically.

Portugal	20150.14	1452
Finland	20765.93	685
Channel Islands	20360.83	747
Iceland	1077	395
Italy	17298.96	758

Task 4: Store and Retrieve Data Using SQL

Step 17: Create SQLite Database

```
con <- dbConnect(
  SQLite(),
  "retail_sales.db"
)

# Check connection
dbListTables(con)

# Store final dataset in SQLite
dbWriteTable(
  con,
  "retail_sales",
  retail_final,
  overwrite = TRUE
)

# Check tables
dbListTables(con)

# Check number of rows in SQLite table
dbGetQuery(
  con,
  "SELECT COUNT(*) AS total_rows FROM retail_sales"
)

# View first 5 records from SQLite
dbGetQuery(
  con,
  "SELECT * FROM retail_sales LIMIT 5"
)
```

A tibble: 5 × 3

Country	Total_Revenue	Number_of_Transactions
<chr>	<dbl>	<int>
United Kingdom	7868408.7	349203
Netherlands	329130.8	2359
EIRE	287260.9	7226
Germany	233804.1	9025
France	203628.3	8326

A tibble: 5 × 3

Country	Total_Revenue	Number_of_Transactions
<chr>	<dbl>	<int>